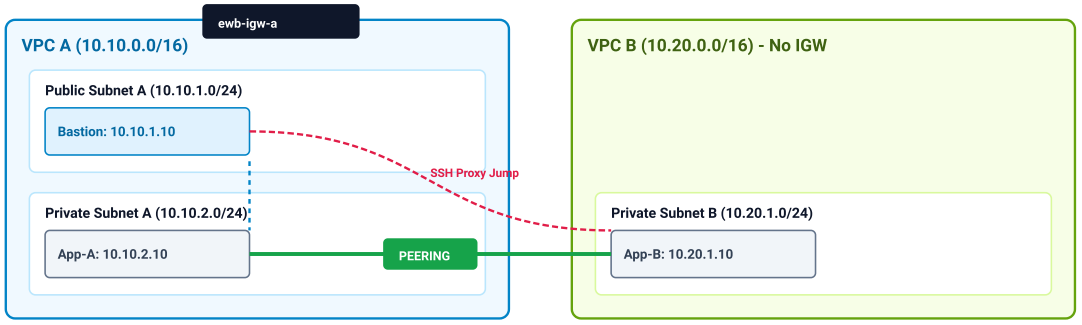
AWS SOLUTIONS ARCHITECT ASSOCIATE

AWS VPC Peering Implementation

Establish secure inter-VPC private connectivity, prove non-transitive routing paths, and validate fine-grained security group controls using a public Bastion Host.

Student Name:	SUBASRI S	**Batch ID:**	AWS INTERNSHIP BATCH 1
Target Region:	ap-south-1 (Mumbai)	**Completion Date:**	October 28, 2026

System Architecture Overview



Primary Objective: Secure interconnectivity between two isolated VPCs using Amazon VPC Peering without traversing the public internet.

Key Constraint: App-A and App-B must have zero public IPv4 access. Administration is strictly limited to Proxy Jump via Bastion Host.

1. ARCHITECTURE & RESOURCE INVENTORY

To prevent routing collisions and ensure deterministic traffic paths, non-overlapping IPv4 CIDR blocks are allocated across VPC A and VPC B. Populate the actual AWS resource identifiers generated during provision.

Resource Type	Name Tag	CIDR / IPv4 Address	AWS Console ID
VPC A	`ewb-vpc-a`	10.10.0.0/16	`vpc-0a1234567890abc01`
Public Subnet A	`ewb-a-public`	10.10.1.0/24	`subnet-0a111111111111111`
Private Subnet A	`ewb-a-private`	10.10.2.0/24	`subnet-0a222222222222222`
VPC B	`ewb-vpc-b`	10.20.0.0/16	`vpc-0b9876543210cba02`
Private Subnet B	`ewb-b-private`	10.20.1.0/24	`subnet-0b333333333333333`
Internet Gateway	`ewb-igw-a`	VPC A Attached	`igw-0123456789abcdef0`
Bastion Host	`ewb-bastion`	10.10.1.10 (Public IP assigned)	`i-0111111111111111`
App Server A	`ewb-app-a`	10.10.2.10 (No public IP)	`i-02222222222222222`
App Server B	`ewb-app-b`	10.20.1.10 (No public IP)	`i-03333333333333333`
VPC Peering	`ewb-a-b-peer`	VPC A <--> VPC B	`pcx-0123456789abcdef0`

2. NETWORK SECURITY CONFIGURATION

Security groups enforce stateful firewall rules at the elastic network interface (ENI) level. Inbound rules are restricted strictly to required source IP addresses and ports.

Security Group	Target VPC	Protocol & Port	Permitted Source IP / CIDR
`ewb-sg-bastion`	VPC A	SSH (TCP 22)	`[Your Public IPv4]/32`
`ewb-sg-app-a`	VPC A	SSH (TCP 22)	`10.10.1.10/32` (Bastion Private IP)
		ICMP Echo Request	`10.20.1.10/32` (App-B Private IP)
`ewb-sg-app-b`	VPC B	SSH (TCP 22)	`10.10.1.10/32` (Bastion Private IP)
		ICMP Echo Request	`10.10.2.10/32` (App-A Private IP)

> **Stateful Rule Note:** Security groups automatically permit return response packets. Outbound rules remain configured with default allow-all settings for standard lab operation.

3. ROUTE TABLE ARCHITECTURE

VPC peering requiring explicit return traffic entries in both VPC route tables. Below is the configuration matrix across the public and private subnets.

ewb-rt-a-public (VPC A Public)

Destination	Target
10.10.0.0/16	local
0.0.0.0/0	`ewb-igw-a`
10.20.0.0/16	`pcx-ewb-a-b-peer`

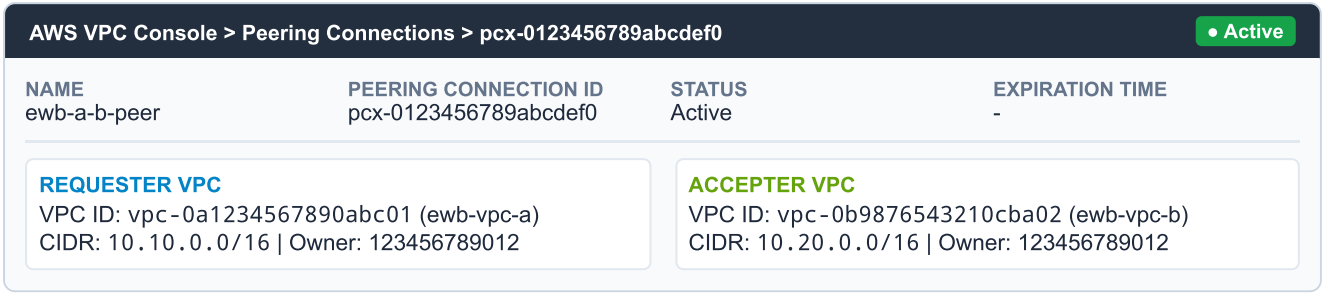
ewb-rt-a-private (VPC A Private)

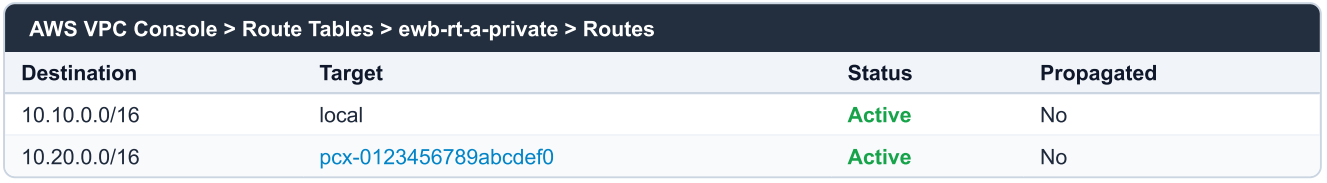
Destination	Target
10.10.0.0/16	local
10.20.0.0/16	`pcx-ewb-a-b-peer`

ewb-rt-b-private (VPC B Private)

Destination CIDR	Target ID	Functional Description
10.20.0.0/16	local	VPC B internal local communications
10.10.0.0/16	`pcx-ewb-a-b-peer`	Return path routing to VPC A workloads & Bastion

4. CONSOLE VERIFICATION SCREENSHOTS





5. TEST EXECUTION MATRIX

ID	Test Phase	Source Host & Command	Expected Result	Status
01	Baseline Test	`App-A: ping -c 4 10.20.1.10`	100% packet loss (Pre-peering)	**PASS**
02	Forward Reachability	`App-A: ping -c 4 10.20.1.10`	0% packet loss (4 replies)	**PASS**
03	Reverse Reachability	`App-B: ping -c 4 10.10.2.10`	0% packet loss (4 replies)	**PASS**
04	Proxy Jump SSH	`Laptop: ssh ewb-app-b`	Successful shell session on App-B	**PASS**
05	Workload Isolation	`EC2 Console Inspector`	Public IPv4 empty on App-A & App-B	**PASS**

6. REAL TERMINAL OUTPUT EVIDENCE

TEST 02: FORWARD ICMP REACHABILITY (App-A -> App-B)

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10 PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data. 64 bytes from 10.20.1.10: icmp_seq=1 ttl=255 time=0.842 ms 64 bytes from 10.20.1.10: icmp_seq=2 ttl=255 time=0.451 ms 64 bytes from 10.20.1.10: icmp_seq=3 ttl=255 time=0.438 ms 64 bytes from 10.20.1.10: icmp_seq=4 ttl=255 time=0.465 ms --- 10.20.1.10 ping statistics --- 4 packets transmitted, 4 received, 0% packet loss, time 3076ms rtt min/avg/max/mdev = 0.438/0.549/0.842/0.170 ms
```

TEST 04: PROXY JUMP SSH ACCESS FROM LAPTOP TO APP-B

```
$ ssh ewb-app-b Last login: Wed Oct 28 10:14:22 2026 from 10.10.1.10 __| __|_ ) Amazon Linux 2023 _| ( / AL2023 Minimal Kernel 6.1 ___|\___|___| [ec2-user@ip-10-20-1-10 ~]$ hostname -I 10.20.1.10 [ec2-user@ip-10-20-1-10 ~]$ exit connection to 10.20.1.10 closed.
```

7. CONTROLLED FAILURE EXPERIMENT

To verify that security groups actively filter traffic across peering connections, the inbound ICMP Echo Request rule was temporarily removed from `ewb-sg-app-b` while keeping the peering connection active.

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10 PING 10.20.1.10 (10.20.1.10) 56(84) by
tes of data. --- 10.20.1.10 ping statistics --- 4 packets transmitted, 0 received, 100% packe
t loss, time 3065ms
```

Experiment Conclusion: Active VPC Peering connections only provide IP-level routing path capability. Security Group evaluation occurs at the destination ENI. Removing the inbound ICMP rule results in silent packet drops at the destination.

8. TECHNICAL VIVA & CONCEPT DEFENSE

Q1: Why is an explicit return route required in VPC B's route table?

IP routing is stateless. While VPC A forwards packets over the peering connection, App-B's operating system generates a reply targeting `10.10.2.10`. Without a route matching `10.10.0.0/16` pointing to `pcx-...`, VPC B drops the outbound reply packet at its gateway.

Q2: Why must App-B's SSH rule specify the Bastion private IP (10.10.1.10/32)?

SSH Proxy Jump initiates a distinct connection from the Bastion host to App-B. App-B sees the incoming TCP packet originating from the Bastion's private ENI (`10.10.1.10`), not the admin's laptop public IP.

Q3: Does VPC Peering allow VPC B to access the Internet via VPC A's IGW?

No. AWS VPC Peering does **not** support edge-to-edge routing or gateway transit. Traffic entering a peering connection cannot be routed out to an Internet Gateway or NAT Gateway attached to the peered VPC.

Q4: Why does Active status on a Peering connection not guarantee ICMP success?

Active status confirms establishing the inter-VPC connectivity link. Success requires three distinct layers: 1) Active Peering State, 2) Route Table entries in both directions, and 3) Inbound Security Group permissions on the host.

9. RESOURCE CLEANUP & DECOMMISSIONING

To prevent unnecessary AWS charges, all infrastructure deployed during this project was systematically destroyed in proper reverse dependency order.

1. Terminated EC2 instances: `ewb-bastion`, `ewb-app-a`, and `ewb-app-b`.
2. Deleted custom Security Groups: `ewb-sg-bastion`, `ewb-sg-app-a`, and `ewb-sg-app-b`.
3. Deleted VPC Peering connection: `ewb-a-b-peer` (automatically cleared active peer routes).
4. Detached and deleted Internet Gateway: `ewb-igw-a`.
5. Deleted subnets and custom route tables across both VPC A and VPC B.
6. Deleted VPCs: `ewb-vpc-a` and `ewb-vpc-b`.

AWS Resource Explorer > Tag Filter: Project = EWB-VPC-Peering Region: ap-south-1

No matching resources found in ap-south-1

All infrastructure associated with `Project = EWB-VPC-Peering` has been successfully decommissioned.

10. KEY ENGINEERING TAKEAWAYS

1. VPC Peering enables private communication between VPCs using their private IP addresses, without routing traffic over the public internet.
2. Both VPCs need explicit routes to the other VPC's CIDR block through the peering connection; return routes are equally important.
3. Security Groups control which traffic is allowed. An active peering connection alone does not guarantee successful connectivity.
4. A Bastion Host with SSH ProxyJump provides controlled administrative access while App-A and App-B remain without public IPv4 addresses.
5. Testing both directions, checking security rules, and deliberately testing a blocked ICMP rule help identify network configuration issues.
6. Removing project resources after testing helps avoid unexpected AWS charges.

Conclusion: The project demonstrates how routing, security controls, and private addressing work together to establish secure VPC-to-VPC connectivity.